Université Grenoble Alpes
L3 Informatique

Année 2025-2026

# ALM - Quick 24/10/2025

Durée = 1h
Aucun document autorisé.

***NOTE IMPORTANTE.*** *Pour toutes les questions où il est demandé d'expliquer la réponse, des points seront retirés pour une réponse sans explication.*

**NOM :** ……..……..……..…….. **PRENOM** : ……..

**Numéro de groupe** : .....

**Exercice 1.** *(3 points)*

Soit un processeur 32-bits dont le bus mémoire comprend les signaux suivants :
- Access (à 1 quand le processeur demande un accès mémoire)
- $A_{31}$ $A_{30}$ ... $A_0$ formant le bus d'adresses
- $D_{31}$ $D_{30}$ ... $D_0$ formant le bus de données

On dote ce processeur de plusieurs boîtiers de mémoire, dont un boîtier *b* de 256 Mioctets (pour rappel, 1 Mio correspond à $2^{20}$ octets) dont l'adresse de base est 0x60000000.

**Q1.** Donner en hexadécimal la **taille** de ce boîtier *b* en expliquant comment vous l'obtenez.

**Réponse.** 1 pt

**Q2.** Donner en hexadécimal **l'intervalle** des valeurs des adresses qui correspond à ce boîtier, en expliquant comment vous l'obtenez.

**Réponse.** 0,5 pt

**Q3.** Donner et expliquer **l'équation booléenne** du signal $sel_b$ de sélection de ce boîtier.

**Réponse.** 1,5 pt

**Exercice 2.** *(13,5 points)*

L'application à définir ici est écrite pour une carte (fictive) disposant d'un bouton poussoir, de 4 leds colorées, et d'un périphérique interne générateur de nombres aléatoires (RNG). L'objectif sera que, chaque fois que l'utilisateur appuie sur le bouton poussoir, les leds s'allument selon une valeur déduite de la valeur courante disponible dans le RNG.

Le **bouton poussoir** est connecté au port GPIOA (bit dont le numéro est donné par une macro GPIO_BUTT), et les **leds** sont connectées au port GPIOD (bits dont les numéros sont donnés respectivement par les macros GPIO_G pour vert, GPIO_O pour orange, GPIO_R pour rouge, GPIO_B pour bleu, nombres non contigus).

Sur cette carte fictive, chaque port contient 3 registres :
- registre MODE de 32 bits pour la configuration des pins, qui peut être lu et écrit
- registre d'entrée INPUTD de 16 bits, accessible uniquement en lecture,
- registre de sortie OUTPUTD de 16 bits, qui peut être lu et écrit.

et les macros suivantes sont fournies :

```
#define MODEPTR_A ((volatile uint32_t *) 0x80000000)
#define INPUTDPTR_A ((volatile uint16_t *) 0x80000004)
#define OUTPUTDPTR_A ((volatile uint16_t *) 0x80000008)

#define MODEPTR_D ((volatile uint32_t *) 0x80010000)
#define INPUTDPTR_D ((volatile uint16_t *) 0x80010004)
#define OUTPUTDPTR_D ((volatile uint16_t *) 0x80010008)
```

Les fonctions C à écrire ci-dessous utiliseront ces macros.

**Q1.** Dans le registre MODE, la configuration des pins se fait comme suit : 10 pour sortie, 01 pour entrée, et 00 pour inutilisée (cas par défaut).
Pour chaque registre MODE du GPIOA et MODE du GPIOD, donner **l'expression** qui devra être utilisée pour la configuration des pins afin d'utiliser (seulement) le bouton poussoir et les leds, en **expliquant** comment vous l'obtenez.

**Réponse.** 1,5 pt

**Q2.** Ecrire une **fonction *get_bp*** qui renvoie l'état du bouton poussoir (elle renvoie 1 pour enfoncé ou 0 pour relâché, sur un *int*).

**Réponse.** 1 pt

**Q3.** Ecrire une **fonction *wait_bp*** qui attend que le bouton poussoir soit enfoncé (i.e. elle se termine lorsque l'état du bouton poussoir est passé par relâché puis à nouveau enfoncé).

**Réponse.** 2 pts

**Q4.** Ecrire une **fonction *allume_leds***

```
void allume_leds (char green, char orange, char red, char blue);
```

qui prend en paramètre quatre "flags" qui indiquent respectivement si on veut allumer ou éteindre la led correspondante (0 pour éteindre, 1 pour allumer) et qui provoque l'allumage des leds sans modifier les autres pins du port correspondant.

**Réponse.** 2 pts

Les macros suivantes sont fournies pour l'accès aux registres du RNG :

```
#define RNG_CTRL (*(volatile uint32_t *)(0x40025000))
#define RNG_STAT (*(volatile uint32_t *)(0x40025004))
#define RNG_DATA (*(volatile uint32_t *)(0x40025008))
```

et d'autre part les fonctions suivantes sont fournies :

`int check_ready();` qui renvoie 1 si un nouveau nombre est disponible, et 0 sinon,
`int check_no_error();` qui renvoie 1 s'il n'y a pas eu d'erreur de génération, et 0 sinon.

**Q5.** Nous souhaitons mettre en place une solution qui permettra à l'application d'être avisée via une **interruption** chaque fois qu'un nouveau nombre aléatoire sera disponible.

L'ISR pour le RNG s'appelle *RNG_isr*. Nous l'utiliserons pour récupérer le nouveau nombre et le placer dans une variable globale *value*, pourvu que le nombre soit bien disponible et qu'il n'y ait pas eu d'erreur de génération.
La documentation du périphérique indique que, lors de la détection d'une erreur de génération, il faudra remettre à 0 le bit de RNG_STAT dont le numéro est donné par la macro ERR_IS. Elle indique également que, lorsqu'une donnée valide est présente, la lecture de cette donnée provoque automatiquement l'annulation de l'interruption.
Donner la **définition** de ce traitant d'interruption et **commenter** votre code.

**Réponse.** 1,5 pt

**Q6.** Enfin, l'objectif sera de procéder à un allumage des 4 leds en codant la valeur *val* (entre 0 et 15) à utiliser comme suit : étant donné *v* un entier non signé sur 32 bits, et *k* un entier (supposé compris entre 0 et 7), chaque bit *i* de la valeur *val* correspond au bit numéro *k* de l'octet *i* de *v*, *i* compris entre 0 et 3.
Par exemple : *k* est 4, *v* est 01110000110001011010100001110 => *val* est 1010

**(a)** Ecrire **l'expression** permettant de consulter le bit numéro *k* de l'octet *i* d'un entier 32 bits *v*, par exemple on obtiendra 0x10000000 pour l'exemple ci-dessus, avec *i*=3 et *k*=4

**Réponse.** 1 pt

**(b)** En utilisant la réponse à la question (a), écrire une **fonction *calcule_cle***

```
int calcule_cle(int k, uint32_t v);
```

qui calcule et renvoie un entier (compris entre 0 et 15) dont chaque bit *i* correspond au bit numéro *k* de l'octet *i* de *v*

**Réponse.** 2 pts

**Q7.** Dans le programme ci-dessous, **compléter la boucle infinie** afin que, chaque fois que l'utilisateur appuie sur le bouton poussoir, les leds soient allumées selon une valeur *val* calculée grâce à la fonction *calcule_cle*, en utilisant la dernière valeur 32 bits générée par le RNG. Pour cet allumage, on utilisera la fonction *allume_leds* de la question 4.
Les bits de la valeur *val* seront interprétés, du poids fort au poids faible comme : allumage led verte, orange, rouge, bleue.

```
uint32_t value = 0;

int main (void) {
  int val; // pour stocker le resultat de calcule_cle
  int k = 4;  // pour le 1er parametre de calcule_cle
  coldstart();
  configure_ports();  // configuration des GPIO
  init_rng();   // initialisation du RNG
  while(1) {
    // à compléter ici
    ...
  }
  return 0;
}
```

**Réponse.** 2,5 pts

**Exercice 3.** *(3,5 points)*

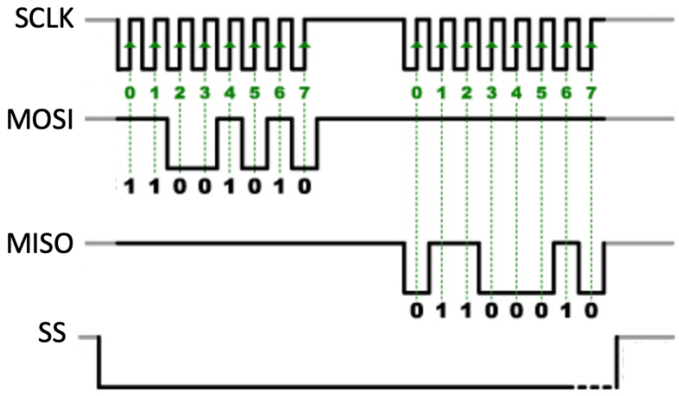
Considérons un système dans lequel un microcontrôleur (maître) est connecté à plusieurs esclaves par une **interface SPI**. L'interface permet l'échange de données sur 8 bits, le bit R/W est à 1 pour une lecture, à 0 pour une écriture.

**Q1.** Pour **chaque signal** mis en jeu dans une transmission via interface SPI, expliquer le **rôle** de ce signal et s'il est **affecté par le maître et/ou par l'esclave** lors des transmissions.

---

**Réponse.** 1 pt

Nous considérons maintenant le chronogramme d'un échange ci-dessous :




**Q2.** Expliquer précisément **comment l'échange s'est déroulé** selon les rôles des différents signaux, et s'il s'agit d'une lecture ou d'une écriture.

---

**Réponse.** 1,75 pt

**Q3.** Quelle est la **destination** de cet échange ? Expliquer et donner la **valeur** en hexadécimal.

**Réponse.** 0,5 pt

**Q4.** Quelle est la **donnée** échangée ? Expliquer et donner la **valeur** en hexadécimal.

**Réponse.** 0,25 pt